กลุ่ม Phoebe

อย่าเอาเงินชั้นไป



Scam Call Alert

แจ้งเตือนสายมิจฉาชีพ จากสิ่งที่ปลายสายพูด

สมาชิกกลุ่ม



ภูดิสพัฒน์ ศรีสุพรรณ์

6620503535



คงยุทธ คล่องแคล่ว

6620501303

สิ่งที่จะนำเสนอ

01
Problem

02
System

03
Datasets

04
Models

05
Evaluation

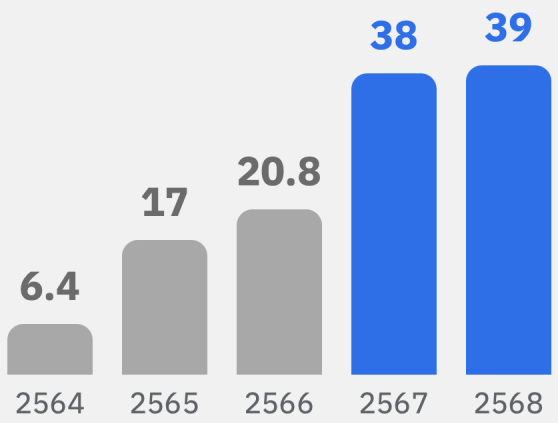
06
Application

07
Conclusion

Demo

เบอร์เปลี่ยนได้ แต่บทหลอกยังเหมือนเดิม

สายหลอกลวงในไทย (ล้านสาย) **×6 ใน 4 ปี**



Whoscall รายงานประจำปี 2564–2568 [12]

ความเสียหายจากคดีออนไลน์

38,105

ล้านบาท

636,482 คดี เฉลี่ย 976 คดีต่อวัน

1 ม.ค. 68 – 13 ต.ค. 69

ประเภทคดีที่พบมากที่สุด

สินค้าและบริการ	**62%**
เสนอผลประโยชน์	**15%**
การจ้างงาน	**8%**
แอบอ้างเป็นบุคคลอื่น	**8%**
การเงินและการลงทุน	**4%**

วิธีเดิมมี Pain Point บล็อกเบอร์ได้ก็ต่อเมื่อมีคนรายงานแล้ว

แนวคิดเรา ใช้ Machine Learning ช่วยฟังว่าปลายสายพูดอะไร แล้วเตือนระหว่างคุย

มองปัญหานี้เป็นโจทย์ ML problem

Scam / Fraud Call Classification

จำแนกสายโทรเข้า จากข้อความที่ได้ยินในสาย ว่าเป็น Scam (สายหลอกลวง) หรือ Not Scam (สายปกติ)

1. INPUT

ข้อความจากเสียงในสาย

เสียงถูกถอดเป็นตัวหนังสือ (STT)

2. MODEL

โมเดล NLP ที่เข้าใจภาษาไทย

เรียนรู้จากตัวอย่างสายหลอกและสายปกติ

3. OUTPUT

Scam / Not Scam

จากผลการ predict ของโมเดล (0...1) เกิน 0.5 ถือว่าเป็น Scam

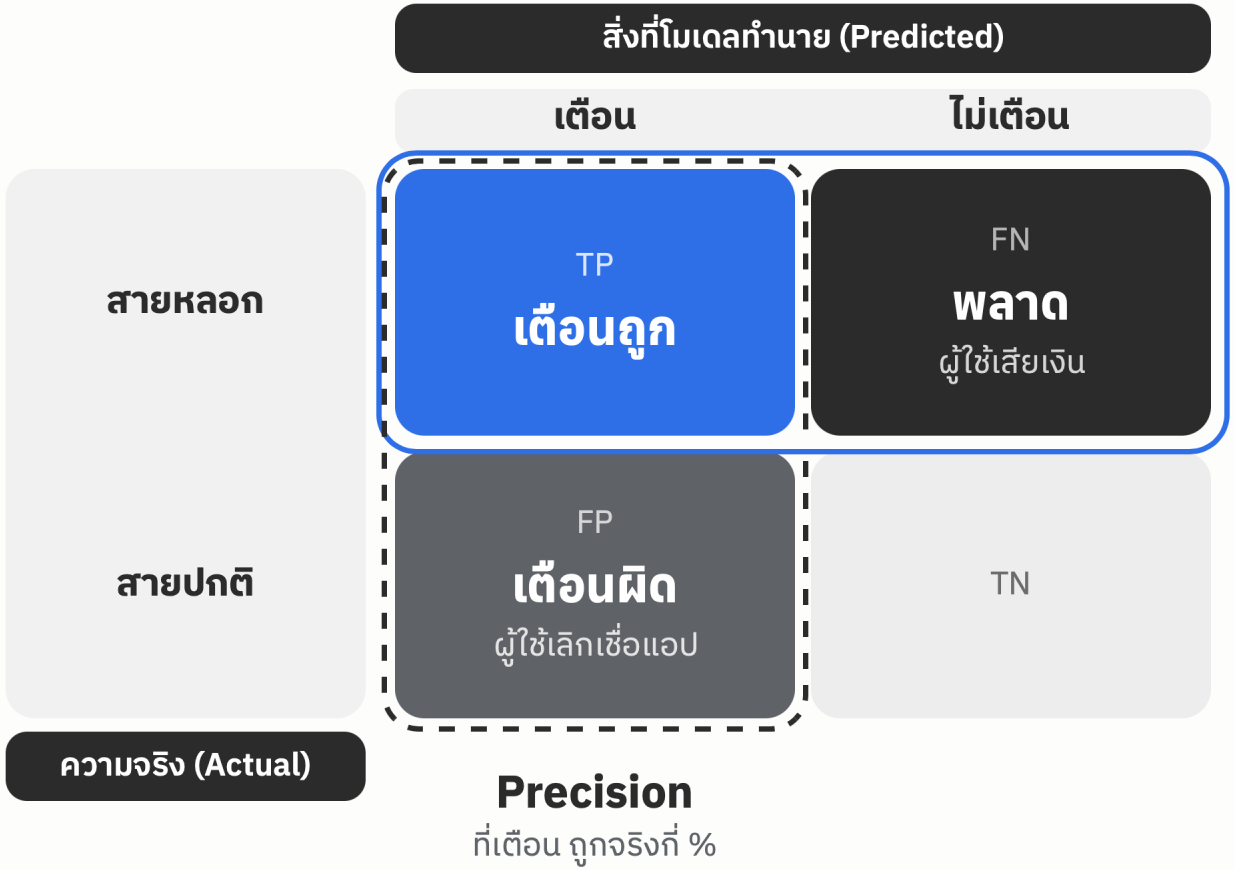
4. ACTION

เตือนผู้ใช้

แสดงแถบเตือนระหว่างสายว่ามีความเสี่ยง

ใช้ metric **Scam F1-Score**: จับสายหลอกให้ได้ และไม่เตือนสายปกติจนผู้ใช้รำคาญ

ทำไมวัดด้วย Scam F1



F1-Score

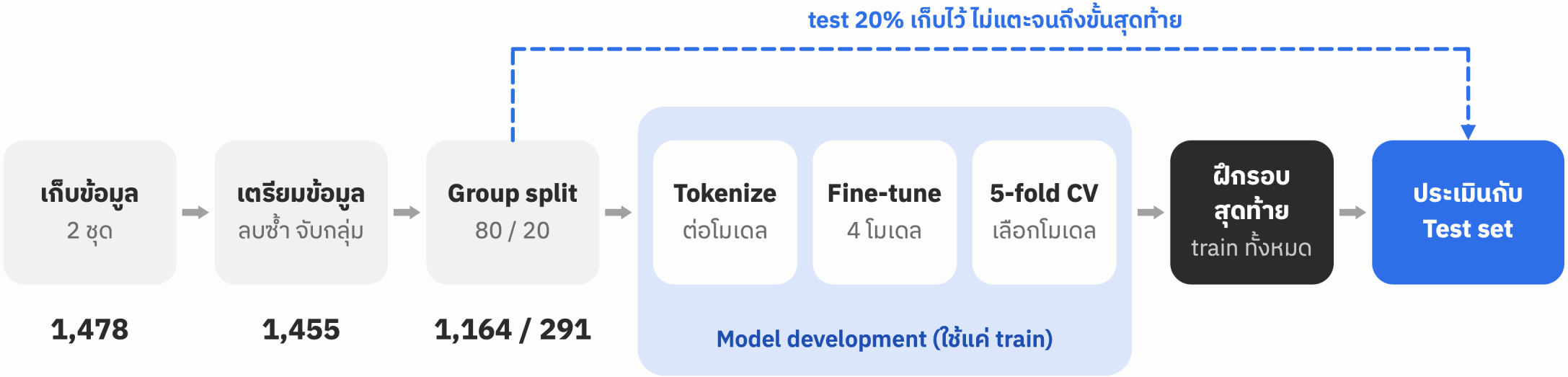
$2 \times P \times R / (P + R)$

ต่ำทันที ถ้าตัวใดตัวหนึ่งต่ำ

สถาปัตยกรรมระบบ



Development Pipeline



Datasets

1. Scam Dialogue Thai Translated

บทสนทนา

1,280

Scam 640 / Not Scam 640

สังเคราะห์จากบทสนทนาภาษาอังกฤษ **BothBosu/scam-dialogue**

แปลเป็นไทยด้วย Claude Sonnet 4.6

มี label เป็น Scam / Not Scam อยู่แล้ว

เผยแพร่บน Hugging Face โดย thirdExec

ตัวอย่าง Scam

"นี่คือเจ้าหน้าที่จอห์นสันจากสำนักงานประกันสังคม"
"หมายเลขประกันสังคมของคุณถูกระงับเนื่องจากมีกิจกรรมที่น่าสงสัย"

ตัวอย่าง Not Scam

"ฉันแค่อยากจะยืนยันว่าพัสดุของฉันจะถูกจัดส่งในวันนี้ประมาณเวลา 14.00-17.00 น."
"มีอะไรอีกไหมที่เราสามารถช่วยคุณได้?"

Datasets

2. Thai Call Center Call Log

บทสนทนา

198

Scam 73 / Not Scam 125

หมวดในแหล่งข้อมูล (จำนวน) → label ที่ใช้

Scam (73)	Scam
General (49)	Not Scam
Official (37)	Not Scam
Verification-call (26)	Not Scam
Unknow (13)	Not Scam

ทุกหมวดที่ไม่ใช่ Scam นับเป็น Not Scam

ตัวอย่าง Scam

"พนักงานไปรษณีย์ โทรมาแจ้งว่าพัสดุของคุณถูกอายัด"

ตัวอย่าง Not Scam

"หวัดดี เราคิดถึงแก"
"คิดถึงเหมือนกัน"
"ไปเจอกันวันเสาร์ไหม"

Synthetic Data นี้เชื่อถือได้แค่ไหน

BOTHBOSU (2024) [4]

Llama-3-70B เขียน บทสนทนาอังกฤษ

1,600 บทสนทนา

→

THIRDEXEC (2026) [1]

Claude Sonnet 4.6 แปลเป็นไทย

ผ่าน distilabel และมี pipeline.yaml

→

ชุดหลักของเรา

1,280 บทสนทนาภาษาไทย

แปลเฉพาะ train split (1,280 จาก 1,600)

มีงานวิจัยใช้จริง

PoPETs 2026 ใช้ชุดต้นฉบับนี้ฝึกโมเดลจับ scam [3]

****ข้อพิจารณา****

ยังเป็นบทสนทนาจำลอง ไม่ใช่สายจริง

Data cleaning

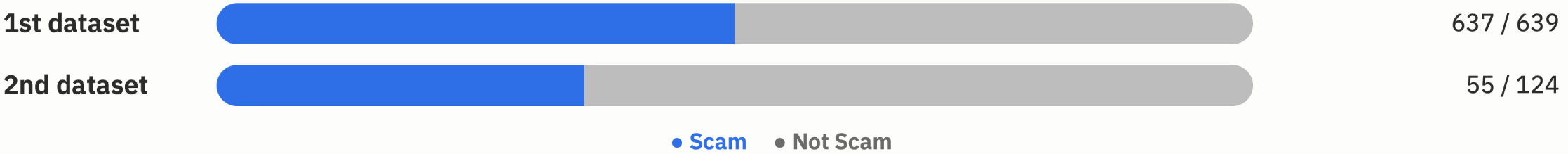
1,478
ข้อมูลดิบ

→

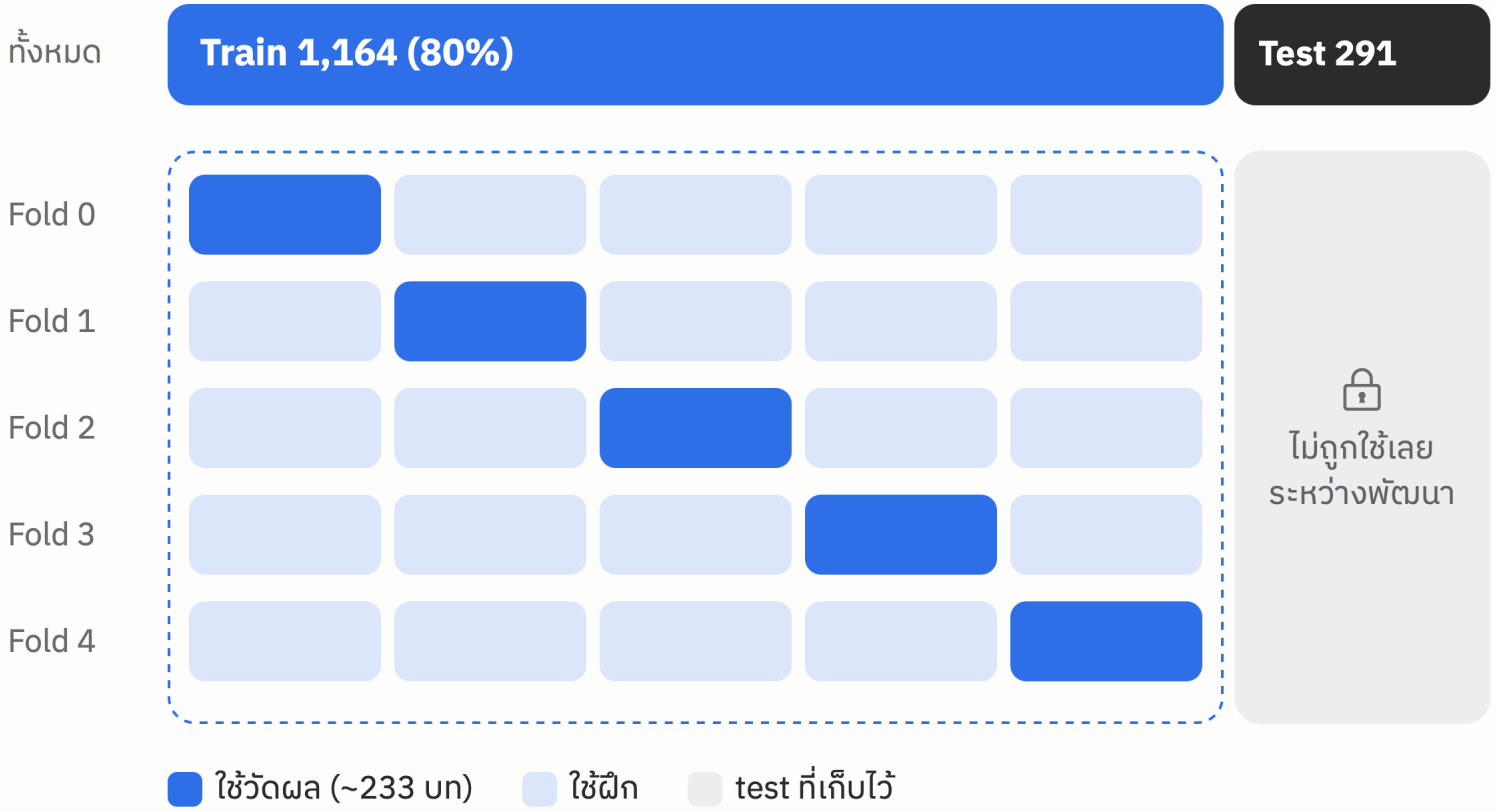
−23
ซ้ำ 20, ถูกตัด 3

→

1,455
พร้อมใช้



แบ่ง Train / Test และ 5 Folds



แยก test ก่อนทุกอย่าง
เก็บไว้วัดครั้งสุดท้าย ให้เหมือนข้อมูล
ที่ไม่เคยเห็น

ใช้ 5-Fold CV
แต่ละรอบฝึก 4 fold + validate กับอีก 1 fold

ทุกโมเดลใช้ fold ชุดเดิม
จึงเทียบกันได้ตรง ๆ

ทำไมใช้ StratifiedGroupKFold

แบ่งข้อมูลเป็น 5 ส่วน (fold) สลับกันเป็นชุดตรวจ และไม่ให้มี data leakage

1 **Group**

จับกลุ่มบทที่เกือบเหมือนกัน

พบ 14 กลุ่ม รวม 37 บท ที่เหลือเป็นบทเดี่ยว

2 **แยก Test 20%**

ทั้งกลุ่มอยู่ train หรือ test ฝั่งเดียว

test 291 บท = ชุดหลัก 255 + ชุดเสริม 36 คงสัดส่วนแหล่งและ label

3 **Stratified · 5 folds**

แบ่ง train เป็น 5 ส่วนสลับกันตรวจ

แต่ละ fold มี Scam / Not Scam ในสัดส่วนเดิม และไม่แยกกลุ่มข้าม fold

02 - DATASETs/scripts/4.split.py

```
strata = [r["source_dataset"] + ":"
          + r["label"] for r in records]
splitter = StratifiedGroupKFold(
  n_splits=5, shuffle=True,
  random_state=67)

# fold แรก = test 20%
train_idx, test_idx = next(
  splitter.split(records, strata, groups))

# 5 folds จาก train อย่างเดียว
for fold, (_, val) in enumerate(
  splitter.split(train, labels, train_groups)):
```

เตรียมข้อความก่อนเข้าโมเดล

ตอนฝึก (มีป้ายกำกับฝ่ายสนทนา)

```
Caller: …
Receiver: …
```

ตอนวัดผล / ในแอป (ไม่มีป้าย)

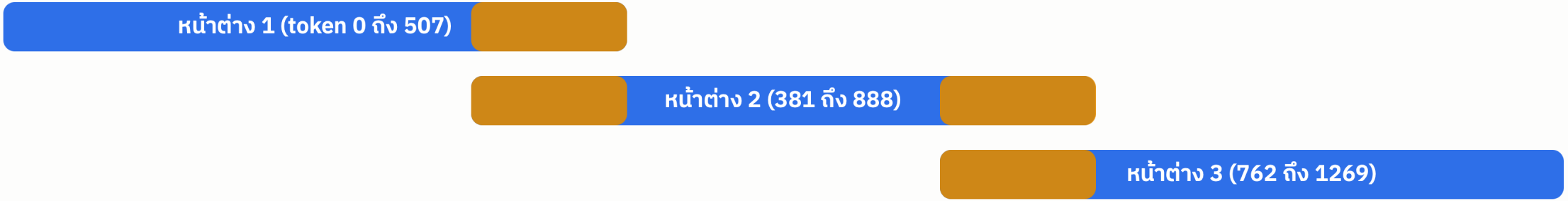
```
… …
… …
```

***ข้อจำกัดของโมเดล* บทสนทนาที่ยาวเกิน 510 tokens จนท้ายถูกตัด**



แก้ input ที่ยาวเกิน token length ของโมเดล

บทสนทนายาวถูกแบ่งเป็น window ละ 508 tokens ซ้อนกัน 127 tokens เพื่อไม่ให้ประโยคขาดตรงรอยต่อ



1

อ่านทีละ window

508 tokens + 2 special tokens = 510 ตามขีดจำกัดของโมเดล

2

ได้คะแนนต่อ window

แต่ละหน้าต่างให้ logit ของ Not Scam และ Scam

3

รวมเป็นคะแนนเดียว

เอาค่า max ของแต่ละคลาสจากทุกหน้าต่าง แล้วคิด loss ครั้งเดียวต่อบทสนทนา

งานวิจัยที่สนับสนุนวิธีนี้ (ทดสอบกับเอกสารยาวและข้อความคลินิก ยังไม่ได้ทดสอบกับบทสนทนาทางโทรศัพท์)

Overlap: ลดการแตกของบริบทที่รอยต่อ window ในเอกสารยาว
Dai et al. (2022) [18]

Max: รวม logit ของ window ด้วย max ได้ผลดีกับข้อความยาว
Gao et al. (2021) [19]

4 Candidate Models

1 / 2: โมเดลภาษาไทย ที่พัฒนาในไทย

VISTEC-depa AI Research Institute (2021) [5]

WangchanBERTa

RoBERTa ที่ฝึกด้วยข้อความภาษาไทยล้วน

Parameters	ข้อมูลฝึก	Vocab
106M	**78.5 GB**	**25k**

สร้างมาเพื่อ ภาษาไทยมีข้อมูลน้อย และโมเดลหลายภาษายังเข้าใจลักษณะภาษาไทยได้ไม่ดี

เราเลือกเพราะ เป็นมาตรฐานของงาน NLP ภาษาไทย

จุฬาฯ (ภาษาศาสตร์) × ClickNext (2023) [6]

PhayaThaiBERT

ต่อยอด WangchanBERTa ให้รู้จักคำต่างประเทศ

Parameters	ข้อมูลฝึก	Vocab
278M	**156.5 GB**	**249k**

สร้างมาเพื่อ อ่านคำอังกฤษที่เขียนปนในข้อความไทยได้ดีขึ้น

เราเลือกเพราะ บทหลอกมักมีชื่อหน่วยงานหรือคำอังกฤษปน

4 Candidate Models

2 / 2: แนวทางรุ่นใหม่ และโมเดลหลายภาษา

โครงการ HPLT ของยุโรป (2025) [7][8]

HPLT GPT-BERT Thai

โมเดลลูกผสม GPT + BERT ภาษาไทยล้วน

Layers	ข้อมูลฝึก	Vocab
24	**HPLT 3.0**	**32k**

สร้างมาเพื่อ ให้มีโมเดลเปิดของแต่ละภาษา ที่เรียนได้ทั้งเติมคำและเดาคำถัดไป

เราเลือกเพราะ เป็นตัวแทนสถาปัตยกรรมรุ่นใหม่

Facebook AI (2020) [9]

XLM-RoBERTa

RoBERTa ที่ฝึกพร้อมกัน 100 ภาษา รวมภาษาไทย

Parameters	ข้อมูลฝึก	Vocab
~270M	**2.5 TB**	**250k**

สร้างมาเพื่อ ใช้โมเดลเดียวได้กับหลายภาษา โดยไม่ต้องฝึกแยกทีละภาษา

เราเลือกเพราะ อยากรู้ว่าโมเดลเฉพาะไทยดีกว่าจริงไหม

เทียบ 4 โมเดลอย่างไรให้ยุติธรรม

1

ข้อมูลชุดเดียวกัน

train 1,164 บทสนทนา แบ่ง 5 folds เหมือนกันทุกโมเดล

2

ฝึกแบบเดียวกัน

AdamW, batch 16, 3 epochs, seed 67

3

วัดด้วยเกณฑ์เดียว

Scam F1 บนข้อความไม่มีป้ายผู้พูด เฉลี่ยจาก 5 folds

4

เลือกแล้วค่อยเปิด test

ตัวที่ดีที่สุดฝึกใหม่บน train ทั้งหมด แล้ววัดบน test ครั้งเดียว

สิ่งที่ต่างกันคือ **ตัวโมเดล**

แผนการทดลอง เทียบกับที่ทำจริง

45 / 75 รอบที่วางแผนไว้

WangchanBERTa (6 config)	● ● ● ● ● × 6
PhayaThaiBERT 3e-5	● ● ● ● ●
HPLT 3e-5	● ● ● ● ●
XLM-R 3e-5	● ● ● ● ●
3 ตัวหลัง (LR 2e-5, 5e-5)	○ ○ ○ ○ ○ × 6 ไม่ได้รัน

1 ช่อง = 1 fold (3 โมเดลหลังรันไม่ครบด้วยข้อจำกัดด้านเวลา และฮาร์ดแวร์)

selection/workflow.py

```
MODELS = ("wangchanberta",
  "phayathaibert", "hplt", "xlmr")
RATES = (2e-5, 3e-5, 5e-5)
FOLDS = range(5)

for model in MODELS:
 for pooling in POOLING[model]:
  for lr in RATES:
   for fold in FOLDS:
    rows.append({…})
if len(rows) ≠ 75:
  raise ValueError(…)
```

ตั้งค่าการฝึกให้เท่ากันทุกโมเดล

Loss **Cross-Entropy**	Optimizer **AdamW**
Learning rate **2e-5, 3e-5, 5e-5**	Warmup **linear → คงที่**
Batch **16 = 4 × 4**	Weight decay / clip **0.01 / 1.0**
Class weight **ไม่ใช้ (สมดุลแล้ว)**	GPU **T4, FP16**

training/trainer.py

```
optimizer = AdamW(groups,
    lr=opt_cfg["learning_rate"],
    betas=(opt_cfg["beta1"], opt_cfg["beta2"]),
    eps=opt_cfg["epsilon"])
scaler = torch.amp.GradScaler(device.type,
    enabled=precision == "fp16")

losses = F.cross_entropy(logits.float(),
    labels, reduction="none")
clip_grad_norm(model.parameters(), max_norm)
```

โมเดลเรียนรู้อย่างไร

ขั้นที่ 1
ทาย
สายนี้น่าจะหลอกกี่ %

→

ขั้นที่ 2
ให้คะแนนความผิด
เรียกว่า Loss

→

ขั้นที่ 3
แก้ให้ดีขึ้น
โดย optimization algorithm

ทำซ้ำหลายรอบ จนทายผิดน้อยลง

$$Loss = -[\ y \cdot \ln(p) + (1-y) \cdot \ln(1-p)\ ]$$

y = คำตอบจริง (สายหลอก = 1, สายปกติ = 0)
p = โอกาสที่โมเดลทายว่าหลอก ถ้า y = 1 จะเหลือ Loss = −ln(p)

ตัวอย่าง: ความสัมพันธ์ของผล predict ของโมเดลกับ loss ของวิธี Cross-Entropy จาก True Positive sample

ทายว่าหลอก 90%	ทายว่าหลอก 50%	ทายว่าหลอก 10%
0.11	**0.69**	**2.30**
ทายถูก ผิดน้อย	**ไม่แน่ใจ**	**มั่นใจแต่ผิด โดนหนักที่สุด**

AdamW ปรับโมเดลอย่างไร

ทำไมเลือก

BERT ต้นฉบับของ Google (2018) ใช้วิธีนี้ ซึ่งเป็นต้นแบบของ WangchanBERTa และ PhayaThaiBERT ที่เราใช้

Devlin et al. (2019)

1

รู้ว่าควรแก้ไปทางไหน

gradient บอกว่าควรปรับ weight แต่ละค่าไปทางไหน เพื่อให้ loss ลดลง

2

ก้าวให้พอดี

ปรับขนาดก้าวให้ weight แต่ละค่าเอง gradient แรงหรือเบา ก็ก้าวใกล้เคียงกัน

3

ดึง weight ให้เล็กลง

ลด weight ลงเล็กน้อยทุกครั้ง (weight decay) ลดการท่องจำข้อมูลฝึก (overfitting)

ตัวอย่าง gradient ลดลง 10 เท่า (0.20 → 0.02) SGD ก้าวลดลง 90% แต่ AdamW ลดแค่ราว 35% (คำนวณประกอบ ไม่ใช่ผลฝึกจริง)

ข้อจำกัด ยังไม่ได้ลองเทียบกับวิธีอื่น จึงบอกได้ว่าเป็นวิธีมาตรฐาน แต่ไม่ได้บอกว่าดีที่สุด

รายละเอียดเพิ่มเติม: Backup B4 (AdamW) B5 (Gradient) B6 (เหตุผลเต็ม)

ผล Cross-Validation



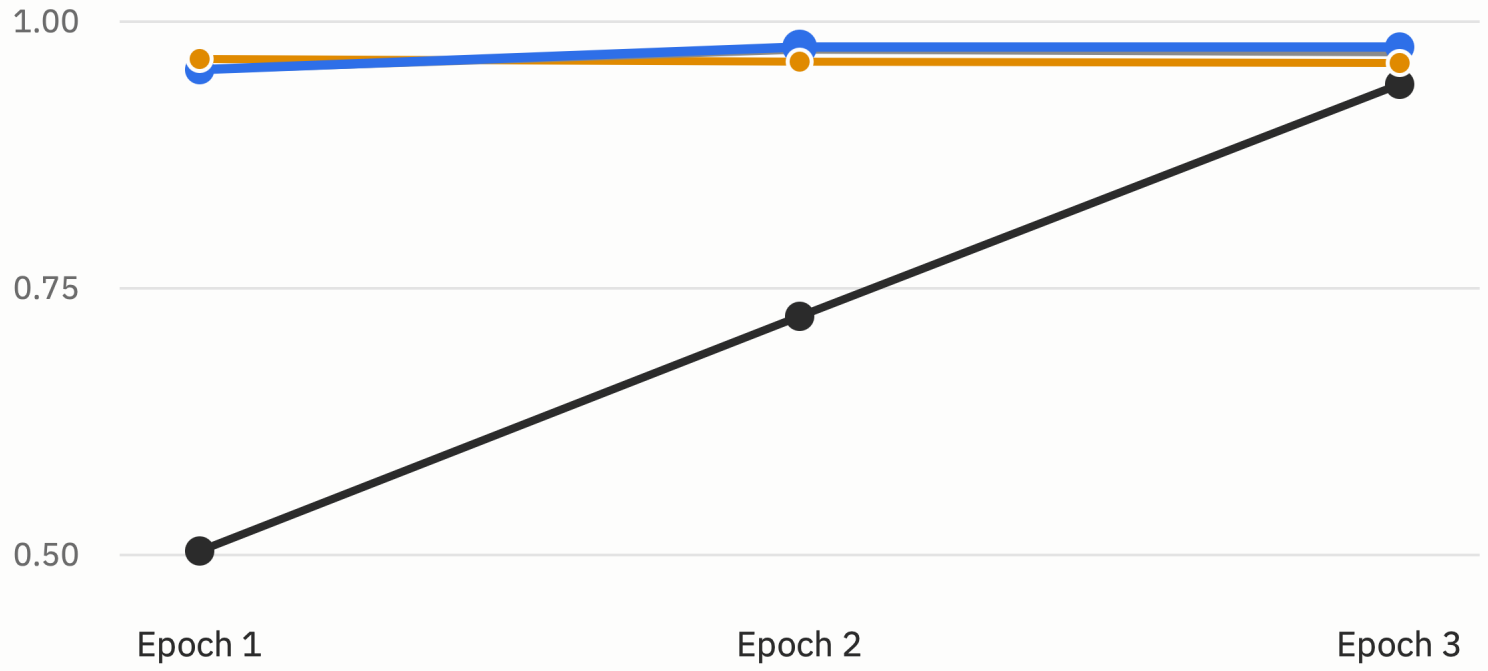
Role-free Scam F1 เฉลี่ย 5 folds ที่ LR 3e-5 (แกนเริ่มที่ 0.90)

ห่างอันดับสอง

0.003

ต่างกันน้อยกว่าความแกว่งระหว่าง fold
จึงยังบอกไม่ได้ว่าเหนือกว่าชัดเจน

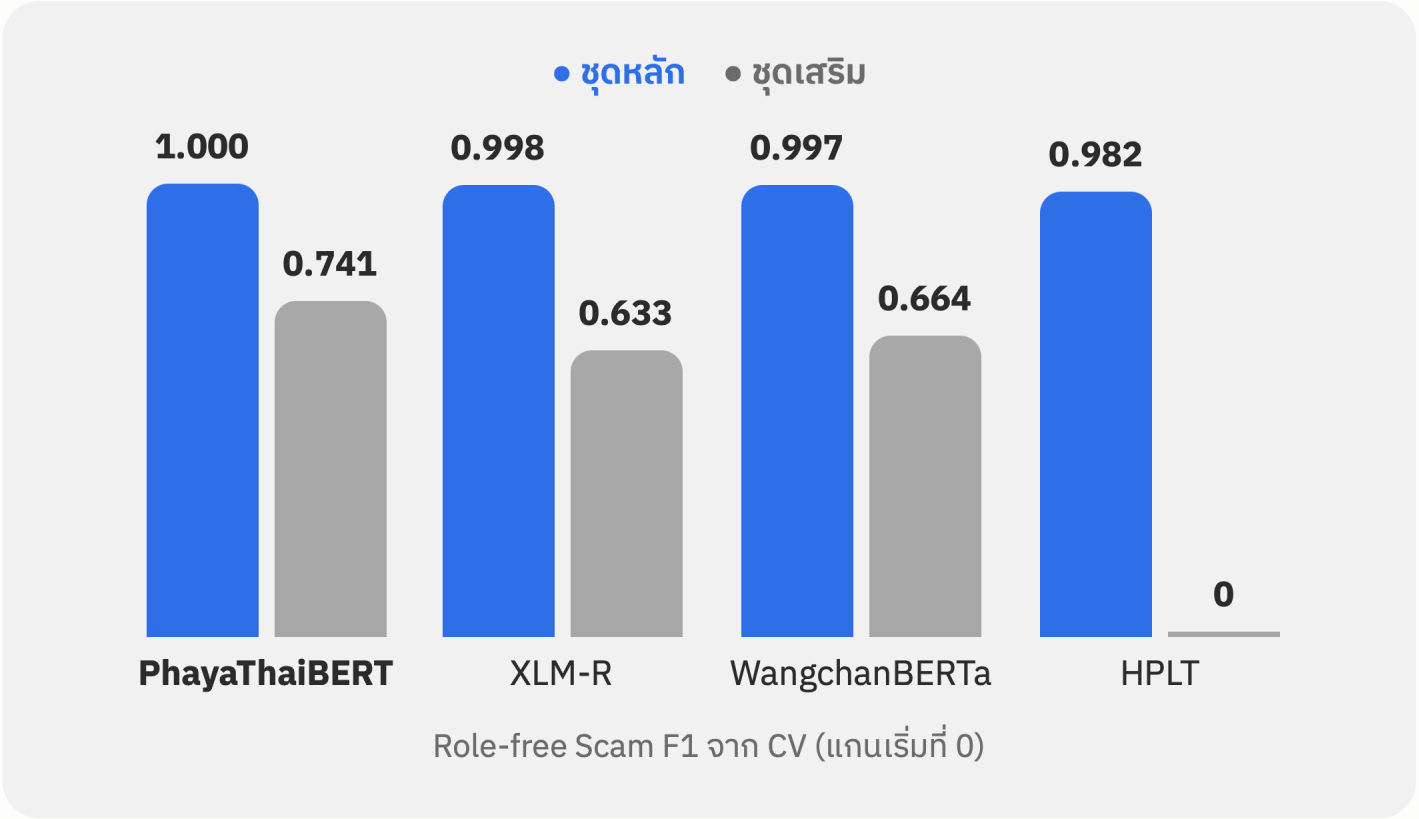
ฝึกกี่ epoch ถึงจะพอ



- **PhayaThaiBERT**
  0.956 → 0.977 → 0.976
- XLM-R
  0.955 → 0.974 → 0.971
- WangchanBERTa
  0.965 → 0.963 → 0.962
- HPLT
  0.504 → 0.724 → 0.941

HPLT ต้องฝึกนานกว่าตัวอื่น ส่วน PhayaThaiBERT ดีที่สุดที่ epoch 2

แยกดูทีละแหล่งข้อมูล



26 / 26

จุดที่ PhayaThaiBERT ทายผิด อยู่ในชุดเสริมทั้งหมด

ชุดเสริมใน train 143 รายการ (FP 17, FN 9)

WangchanBERTa ก็เป็นแบบเดียวกัน: ผิด 39 จุด อยู่ในชุดเสริม 36 (FP 27, FN 9)

โมเดลที่เลือก

PhayaThaiBERT

LR 3e-5, 2 epochs, threshold 0.5

Scam F1 ใน CV	บทสนทนาที่ใช้ฝึกรอบสุดท้าย	เวลาฝึกรอบสุดท้าย (นาที)
0.977	**1,021**	**41.5**

ผลจริงบน Test Set

Scam F1

0.974

Precision **0.978**

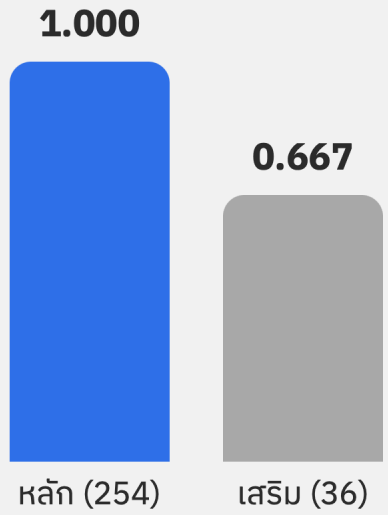
Recall **0.971**

Accuracy **0.976**

Confusion matrix (290 รายการ)

	ทาย ปกติ	ทาย Scam
จริง ปกติ	150	3
จริง Scam	4	133

Scam F1 ตามแหล่งชุดข้อมูล



โมเดลพลาดตรงไหน

4 หลอกจริง แต่ไม่เตือน

“คุณมีพัสดุตกค้าง ที่ต้องรีบจ่ายค่าธรรมเนียมทันที ไม่งั้นของจะโดนทำลาย”

p(Scam) = 0.16

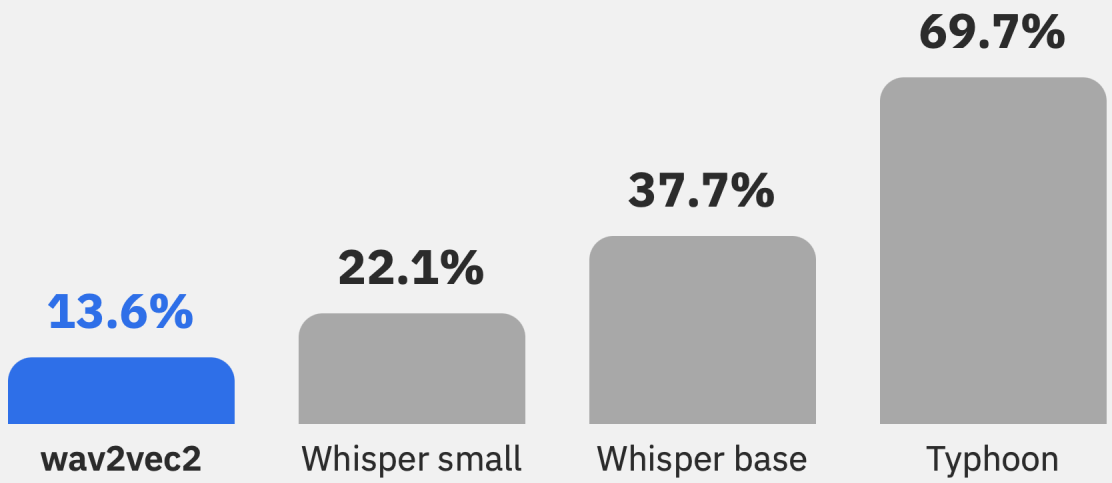
3 สายปกติ แต่เตือน

“โทรจากกรมควบคุมโรค แจ้งว่าคุณเป็นผู้สัมผัสใกล้ชิด COVID-19”

p(Scam) = 0.65

ทั้ง 7 ครั้งมาจากชุดเสริม และเป็นบทสนทนาสั้น ๆ เพียง 5–59 tokens

เลือกตัวถอดเสียง (STT)



CER ยิ่งต่ำยิ่งดี (FLEURS Thai 35 คลิป [11])

ความเร็ว (real-time factor)

0.03×

เสียง 1 นาที ถอดเสร็จในราว 2 วินาที

แม่นและเร็วที่สุด แต่ใช้หน่วยความจำ GPU มากที่สุด

CER คือสัดส่วนตัวอักษรที่ถอดผิด (ทดสอบ 3 รอบ สำเร็จครบ 420 / 420 คลิป)

ระหว่างสาย ควรให้โมเดลอ่านช่วงไหน



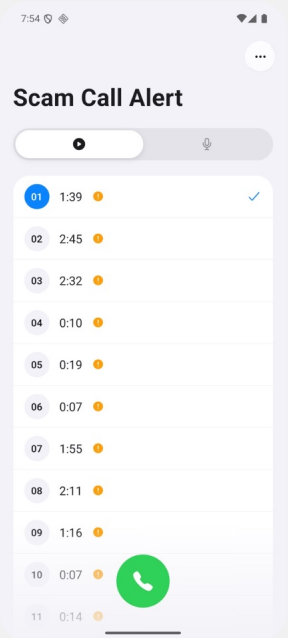
Scam F1 จาก 8 recordings (แบบละ 1 ครั้ง)

prototype/service/core.py

```python
if self.approach == "latest":
  predictive = new_speech
elif self.approach == "recent":
  predictive = "".join(
    x["text"] for x in segments[-n:])
else:  # accumulated
  predictive = self.transcript

# ตัดเหลือท้ายสุด ≤ 510 tokens
context, omission = \
  self.context_builder(predictive)
```

ใช้งานจริงเป็นอย่างไร



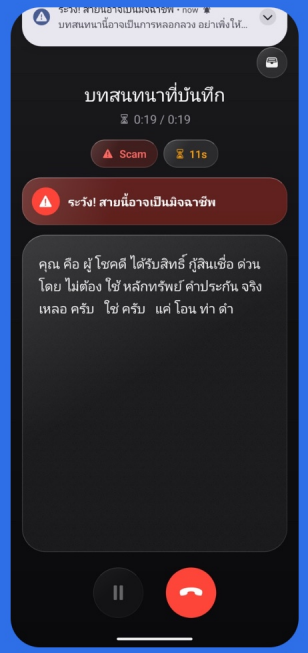
1

เลือกบทสนทนา



2

รับสาย



3

ขึ้นเตือนทันที

เจอ Scam ครั้งแรก แถบเตือนจะค้างไว้ตลอดสาย และทำงานได้แม้แอปอยู่เบื้องหลัง

ทดสอบว่าระบบทำงานได้จริง

รันพร้อมกันได้

STT + โมเดล ใช้หน่วยความจำ GPU ≈ 3.3 GB

สายหลอก → เตือน

เตือนพร้อม notification

สายปกติ → ไม่เตือน

ไมโครโฟนสด

ถอดเสียงและเตือนได้แบบสด

สิ่งที่ยังทำไม่ได้

ข้อมูลเป็นบทสนทนาจำลอง
ยังไม่ได้ลองกับสายจริง

ทดลองได้ไม่ครบแผน
45 จาก 75 รอบ และใช้ seed เดียว

อ่อนกับข้อมูลต่างแหล่ง
ชุดเสริม F1 0.667 บน test

บทสนทนายาวถูกตัดท้าย
30% ของ Scam เกิน 510 tokens

STT วัดบนเสียงอ่าน
ไม่ใช่เสียงโทรศัพท์จริง

ยังไม่มีผลระดับระบบ
รันบน emulator ไม่ใช่มือถือจริง

Conclusion & Future improvements

แยกบทสนทนาหลอกได้ดี

Scam F1 0.974 บน test

ยังพลาดกับบทสนทนาสั้น

และข้อมูลต่างแหล่ง

แอปต้นแบบใช้ได้ครบวงจร

เสียง → ข้อความ → เตือน

ก้าวต่อไป: ลองกับเสียงสนทนาจริง, ฝึกด้วยข้อมูลหลายแหล่ง, วัดผลทั้งระบบ และย้ายโมเดลไปรันบนมือถือ

DEMO



Role-free Scam F1 ทุก configuration (mean ± SD, 5 folds)

Configuration	Epoch 1	Epoch 2	Epoch 3
Wangchan max, 2e-5	0.9623 ± 0.0118	0.9624 ± 0.0091	0.9606 ± 0.0054
Wangchan mean, 2e-5	0.9637 ± 0.0174	0.9519 ± 0.0117	0.9511 ± 0.0138
Wangchan max, 3e-5	0.9654 ± 0.0104	0.9625 ± 0.0120	0.9616 ± 0.0086
Wangchan mean, 3e-5	0.9607 ± 0.0115	0.9543 ± 0.0140	0.9544 ± 0.0098
Wangchan max, 5e-5	0.9636 ± 0.0141	0.9623 ± 0.0138	0.9623 ± 0.0138
Wangchan mean, 5e-5	0.9562 ± 0.0127	0.9542 ± 0.0115	0.9542 ± 0.0081
PhayaThaiBERT 3e-5	0.9556 ± 0.0247	0.9768 ± 0.0129	0.9760 ± 0.0113
HPLT 3e-5	0.5036 ± 0.1788	0.7244 ± 0.2759	0.9412 ± 0.0305
XLM-R 3e-5	0.9552 ± 0.0316	0.9736 ± 0.0057	0.9707 ± 0.0126

ฝึกด้วยชุดหลัก วัดผลกับทั้งสองแหล่ง (ที่มา: selection/cv_results.csv)

Supplemental Scam F1 รายfold (role-free)

Fold	0	1	2	3	4
N (Not Scam / Scam)	20 / 12	22 / 9	18 / 7	23 / 9	16 / 7
PhayaThaiBERT (epoch 2)	0.750	0.667	0.933	0.640	0.714
XLM-R (epoch 2)	0.727	0.625	0.667	0.700	0.444
HPLT (epoch 3)	0.000	0.000	0.000	0.000	0.000

Primary F1

PhayaThaiBERT 1.000 ทุก fold, XLM-R 0.995–1.000, HPLT 0.922–1.000

PhayaThaiBERT pooled confusion

Supplemental [[TN 82, FP 17], [FN 9, TP 35]]

ป้ายผู้พูด เวลาฝึก และการตัดข้อความ

Model	Role-free F1	Marked F1	เวลาฝึก 5 folds	Scam ยาวเกิน budget
PhayaThaiBERT	0.9768	0.9791	97 นาที T4	30.0% (510)
XLM-R	0.9736	0.9718	149 นาที T4+CPU	27.5% (512)
HPLT	0.9412	0.9596	55 นาที T4	1.0% (512)

ป้ายผู้พูด

ถอดป้ายผู้พูดออก คะแนน PhayaThaiBERT เปลี่ยนแค่ −0.002 ± 0.009

เวลาฝึก

รวม I/O และ setup และ hardware ต่างกัน จึงเทียบความเร็วตรง ๆ ไม่ได้

AdamW ทำงานอย่างไร (รายละเอียด)

Loss
คะแนนความผิดของการทาย

Gradient
ทิศทางและแรงที่ควรแก้ เพื่อลด loss

Learning rate (lr)
ขนาดก้าวพื้นฐาน

Weight
ค่าภายในโมเดลที่ถูกปรับ

SGD (แบบธรรมดา)

weight ← weight − lr × gradient

ก้าวตามขนาดของ gradient ตรง ๆ

gradient แรง (0.20) — **ก้าว 100%**

gradient เบา (0.02) — **ก้าว 10%**

gradient เล็ก ก้าวแทบไม่ขยับ

AdamW

ก้าว = lr × gradient เฉลี่ย ÷ ขนาด gradient เฉลี่ย

ปรับขนาดก้าวจาก gradient ที่ผ่านมา

gradient แรง (0.20) — **ก้าว 100%**

gradient เบา (0.02) — **ก้าว 65%**

ก้าวสม่ำเสมอ loss ลดได้ต่อเนื่อง

+ Weight decay (ตัว W) ลด weight ลงเล็กน้อยทุกก้าว ช่วยลด overfitting (การท่องจำข้อมูลฝึก)

Gradient มาจากไหน

ลองขยับ weight นิดเดียว แล้วดูว่า loss เปลี่ยนไปเท่าไร

1 **คำนวณ loss ตอนนี้**

`loss = 2.3050`

2 **ขยับ weight ขึ้นนิดเดียว (+0.001)**

`loss ใหม่ = 2.3041`

3 **gradient = loss ที่เปลี่ยน ÷ ระยะที่ขยับ**

`(-0.0009) ÷ 0.001 = -0.9`

ตัวอย่างสายหลอกจริง (ใช้ทำขั้นที่ 1 ถึง 3 กับแถวล่างสุด)

โมเดลทายว่าหลอก	loss	gradient
90%	0.11	−0.10
50%	0.69	−0.50
10%	2.30	−0.90

ยิ่งทายผิดมาก gradient ยิ่งแรง

ค่าติดลบ คือขยับ weight ขึ้นแล้ว loss ลด

ในโมเดลจริง weight มีหลายร้อยล้านค่า จึงไม่ลองขยับทีละค่า แต่ใช้ backpropagation คำนวณ gradient ทุกค่าพร้อมกัน (PyTorch ทำให้อัตโนมัติ)

โมเดลจำลองมี weight ค่าเดียว ตัวเลขคำนวณเอง ไม่ใช่ผลการฝึกจริง

ทำไมเลือก AdamW (รายละเอียด)

AdamW คือ optimization algorithm ที่ใช้แก้โมเดลทีละนิดระหว่างเรียนรู้

1

เป็นวิธีมาตรฐานของโมเดลภาษา

ใช้กับ BERT ของ Google (2018) ต้นแบบของ WangchanBERTa และ PhayaThaiBERT ที่โครงงานใช้

Devlin et al. (2019)

2

เหมาะกับโมเดลขนาดใหญ่ เพราะแต่ละค่าต้องการก้าวไม่เท่ากัน

Gradient ตัวเลขที่บอกว่าควรปรับ weight ค่านั้นไปทางไหน และมากแค่ไหน เพื่อให้ loss ลด (เหมือนความชันของเนินเขา)

ปัญหา โมเดลมี weight หลายร้อยล้านค่า (PhayaThaiBERT 278 ล้านค่า) แต่ละค่ามี gradient ต่างกันมาก บางค่าแรง บางค่าเบาจิ๋ว

SGD ก้าวตามขนาด gradient ค่าที่เบาแทบไม่ขยับ ค่าที่พุ่งสูงก้าวไกลเกินไป

AdamW ปรับขนาดก้าวให้แต่ละค่าเอง ทุกค่าก้าวพอดี

Zhang et al. (NeurIPS 2020) พบว่า SGD ทำได้แย่กว่ากับโมเดลแบบ attention

ข้อจำกัด ยังไม่ได้ลองเทียบกับวิธีอื่น จึงยังบอกไม่ได้ว่าเป็นวิธีที่ดีที่สุด

จับบทสนทนาที่เกือบเหมือนกันด้วย Cosine similarity

คืออะไร

Cosine similarity

วัดว่าข้อความสองบท "ชี้ไปทางเดียวกัน" แค่ไหน หลังแปลงข้อความเป็นเวกเตอร์ของตัวเลข

$\cos = (A \cdot B) / (\|A\| \|B\|)$

1 = ใช้ชิ้นข้อความชุดเดียวกันในสัดส่วนเดียวกัน
0 = ไม่มีชิ้นข้อความร่วมกันเลย

ไม่ขึ้นกับความยาวบท

ทำไมใช้

เหมาะกับบทสนทนาแม่แบบเดียวกัน

ไม่ต้องตัดคำ ภาษาไทยไม่เว้นวรรคระหว่างคำ จึงใช้ชิ้นตัวอักษร 3–5 ตัวติดกันแทน

จับการแก้เล็กน้อยได้ เปลี่ยนชื่อ เบอร์โทร หรือจำนวนเงิน ชิ้นส่วนใหญ่ยังซ้ำ ค่าจึงสูง

ทำซ้ำได้ เทียบครบทุกคู่ ไม่มีการสุ่ม ผลเหมือนเดิมทุกครั้ง

ทำงานอย่างไร (วิธีของเรา)

1. จัดข้อความ: ปรับรูปแบบขึ้นบรรทัดใหม่ และตัดช่องว่างหัวท้ายเท่านั้น
2. แปลงเป็นเวกเตอร์ **TF-IDF** ของชิ้นตัวอักษร 3–5 ตัว ชิ้นที่มีเฉพาะบางบทได้น้ำหนักมาก ชิ้นที่ทุกบทมีได้น้ำหนักน้อย
3. คำนวณ cosine ของ**ทุกคู่**ใน 2 แหล่ง เก็บคู่ที่ ≥ **0.90** และข้อความไม่เหมือนกันทั้งหมด
4. ตรวจทีละคู่ ยืนยันว่าเกี่ยวข้องกัน แล้วต่อคู่ที่เกี่ยวกันเป็นกลุ่ม

46 คู่

ที่ cosine ≥ 0.90 (ค่า 0.90–0.99)
ยืนยันเกี่ยวข้องทั้งหมด

14 กลุ่ม

รวม 37 บท ส่วนที่เหลือเป็นบทเดี่ยว

0

คู่ที่ label ขัดกัน และคู่ที่ข้ามแหล่งข้อมูล

เทียบ STT 4 ตัว: ความแม่น ความเร็ว และทรัพยากร

โมเดล	CER ↓	RTF ↓	โหลด (วิ)	CPU % เฉลี่ย / สูงสุด	RAM โปรเซส (GB)	GPU memory (GiB)
wav2vec2 Thai (เลือกใช้) PyThaiASR · XLSR-53 Thai · ไม่ใช้ LM	13.55%	0.028	4.1	10 / 26	0.46	3.10
Whisper small OpenAI · multilingual	22.13%	0.272	5.1	30 / 38	0.66	1.38
Whisper base OpenAI · multilingual	37.72%	0.126	2.6	33 / 43	0.42	1.09
Typhoon ASR Real-Time PyThaiASR · RNN-T · ONNX	69.73%	0.089	7.3	263 / 411	4.02	ไม่มี

wav2vec2
CER ต่ำสุดและเร็วสุด แลกกับ GPU memory สูงสุดในกลุ่ม GPU (3.10 GiB)

Whisper
small แม่นกว่า base แต่ช้ากว่า wav2vec2 ราว 10 เท่า (0.272 เทียบ 0.028)

Typhoon
เร็วกว่าเวลาจริง (0.089) แต่ CER สูงสุด และใช้ CPU กับ RAM สูงสุด (4.0 GB)

MacBook Air M2 RAM 8 GB · FLEURS Thai dev 35 คลิป (7.6 นาที) รัน 3 รอบ ถอดสำเร็จ 420 / 420 · เสียงอ่านประโยค ไม่ใช่เสียงโทรศัพท์ · RTF = เวลาถอด ÷ ความยาวเสียง · GPU mem กับ RAM บวกกันไม่ได้

ความยาว input สูงสุดของแต่ละโมเดล (tokens)

โมเดล	ขีดจำกัดของ checkpoint	ที่เราใช้ (รวม special 2)	เมื่อบทยาวเกิน	Scam ใน train ที่ยาวเกิน
WangchanBERTa	510	510 ต่อ window	ตัดเป็น window 508 tokens เลื่อนทีละ 381 (ซ้อน 127) รวมผลด้วย max / mean	40.0% (ถ้าตัดแบบ prefix)
PhayaThaiBERT	510	510	เก็บ 510 tokens แรก ตัดท้าย	30.0%
HPLT GPT-BERT Thai	16,384	512	เก็บ 512 tokens แรก ตัดท้าย	1.0%
XLM-RoBERTa	512	512	เก็บ 512 tokens แรก ตัดท้าย	27.5%

ตอนฝึก (role-marked)

Scam เป็นฝั่งที่ยาว: มัธยฐาน 337–482 tokens จึงเสี่ยงถูกตัดท้ายมากกว่า Not Scam (0–1.0%)

ตอนใช้งาน (role-free)

ไม่มีป้ายผู้พูด สั้นลง: บทยาวสุดใน train 449–491 tokens ไม่เกินขีดจำกัดของโมเดลใด

ข้อควรจำ

HPLT รับได้ยาวกว่ามาก แต่เรากำหนดไว้ที่ 512 ให้เท่ากันและประหยัดทรัพยากร

นับ tokens ของแต่ละ tokenizer รวม special tokens 2 ตัว · เฉพาะ 1,021 บทสนทนาชุดหลักใน train (Scam 510 / Not Scam 511) · ตัวเลขคือสัดส่วนที่ยาวเกินขีดจำกัด

รายการอ้างอิง

[1] thirdExec, Scam Dialogue Thai Translated, Hugging Face, 2026.

[2] Jxxn03X, Thai Call Center Call Log Dataset, Kaggle, 2025.

[3] I. Hossain et al., “AI-in-the-Loop: Privacy Preserving Real-Time Scam Detection…,” PoPETs 2026(1), pp. 87–114.

[4] BothBosu, Scam Dialogue, Hugging Face, 2024.

[5] L. Lowphansirikul et al., “WangchanBERTa,” arXiv:2101.09635, 2021.

[6] P. Sriwirote et al., “PhayaThaiBERT,” ACM TALLIP, 2025 (arXiv:2311.12475).

[7] L. G. G. Charpentier, D. Samuel, “GPT or BERT: why not both?,” BabyLM @ CoNLL, 2024.

[8] S. Oepen et al., “HPLT 3.0,” arXiv:2511.01066, 2025.

[9] A. Conneau et al., “Unsupervised cross-lingual representation learning at scale,” ACL, 2020.

[10] Z. Li et al., “Synthetic Data Generation with LLMs for Text Classification: Potential and Limitations,” EMNLP, 2023.

[11] A. Conneau et al., “FLEURS,” arXiv:2205.12446, 2022.

[12] Whoscall/Gogolook Thailand annual reports 2021–2025, via The Nation (2023, 2025) and Thai Newsroom (2026).

[13] ศปอส.ตร. (ตำรวจไซเบอร์), สถิติความเสียหายสะสม 1 ม.ค. 68 – 13 ต.ค. 69, thaipoliceonline.go.th (เข้าถึง 14 ต.ค. 69).

[14] J. Devlin et al., “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” NAACL, 2019 (Appendix A.3).

[15] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” ICLR, 2019.

[16] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” ICLR, 2015.

[17] J. Zhang et al., “Why are Adaptive Methods Good for Attention Models?,” NeurIPS, 2020.

[18] X. Dai et al., “Revisiting Transformer-based Models for Long Document Classification,” Findings of EMNLP, 2022.

[19] S. Gao et al., “Limitations of Transformers on Clinical Text Classification,” IEEE JBHI, 2021.

[L02] Pattern Recognition L02_Dataset Management and Data Quality (PDF p.49, 53–54, 101)

[L05] Pattern Recognition L05_Model Selection & Hyperparameter Tuning (PDF p.11, 14–15, 35–36)

[L06] Pattern Recognition L06_Experimental Design and System Evaluation (PDF p.6, 12, 23, 35)